Comparators

In Java, **TreeSet** and **TreeMap store elements in sorted order**.

By **default**, they use **natural ordering** (alphabetical for Strings, numerical for Integers).

But **if you want your own sorting rule** (e.g., **reverse order**, or **sort by last name**), you must use a **Comparator**.

The Comparator Interface

```
interface Comparator<T> {
    int compare(T obj1, T obj2);
}
```

compare(obj1, obj2) must return:

- **Negative** if obj1 < obj2
- **Zero** if obj1 == obj2
- **Positive** if obj1 > obj2

Reverse Order Comparator for TreeSet<String>

Normally, TreeSet stores strings in **ascending order**: A B C D E F

But we want **reverse order**: F E D C B A.

Create a custom Comparator:

```
import java.util.*;

// Custom comparator that sorts in reverse

class MyComp implements Comparator<String> {

        public int compare(String a, String b) {

        return b.compareTo(a);  // Notice: b compared to a (not a to b)

        }

}
```

```
class CompDemo {
        public static void main(String args[]) {
        TreeSet<String> ts = new TreeSet<>(new MyComp());
        ts.add("C");
        ts.add("A");
        ts.add("B");
        ts.add("E");
        ts.add("F");
        ts.add("D");
        for (String element : ts)
                System.out.print(element + " ");
        }
}
```

Output:

F E D C B A

Comparator for TreeMap<String, Double> Sorting by Last Name

Suppose you have **names like "John Doe", "Jane Baker"**, etc.

We want to **sort by LAST NAME** instead of FULL NAME.

```
import java.util.*;

// Comparator that sorts by last name

class TComp implements Comparator<String> {

    public int compare(String a, String b) {

    int i, j, k;

    i = a.lastIndexOf(' ');

    j = b.lastIndexOf(' ');
```

```
// Compare based on last name
        k = a.substring(i).compareTo(b.substring(j));
 // If last names are the same, compare full names
        if (k == 0)
                return a.compareTo(b);
        else
                return k;
        }
}
```

```
class TreeMapDemo2 {
    public static void main(String args[]) {
    TreeMap<String, Double> tm = new TreeMap<>(new TComp());

    // Add some accounts
    tm.put("John Doe", 3434.34);
    tm.put("Tom Smith", 123.22);
    tm.put("Jane Baker", 1378.00);
    tm.put("Tod Hall", 99.22);
    tm.put("Ralph Smith", -19.08);
```

```
// Print entries
        for (Map.Entry<String, Double> me : tm.entrySet()) {
                System.out.println(me.getKey() + ": " + me.getValue());
        }
        System.out.println();
        // Update balance for John Doe
        double balance = tm.get("John Doe");
        tm.put("John Doe", balance + 1000);
        System.out.println("John Doe's new balance: " + tm.get("John Doe"));
        }
}
```

Output

Jane Baker: 1378.0

John Doe: 3434.34

Tod Hall: 99.22

Ralph Smith: -19.08

Tom Smith: 123.22

John Doe's new balance: 4434.34

Comparable interface

The **Comparable interface** is used to define the **natural ordering** of objects. It allows objects of a class to be compared with each other so that they can be sorted automatically (for example in TreeSet, TreeMap, or Collections.sort()).

It is present in:

java.lang.Comparable

Syntax

```
interface Comparable<T> {
   int compareTo(T obj);
}
```

The compareTo() method compares **current object with another object**.

Return values:

- 0 → both objects are equal
- positive → current object is greater
- negative → current object is smaller

Example:

```
class Employee implements Comparable<Employee> {
    String name;
    int age;
    Employee(String name, int age) {
        this.name = name;
        this.age = age;
    }
```

```
public int compareTo(Employee e) {   // Natural sorting by age
        return this.age - e.age;
    }
    public String toString() {
        return name + " - " + age;
    }
}
```

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        List<Employee> list = new ArrayList<>();
        list.add(new Employee("John", 30));
        list.add(new Employee("Alice", 25));
        list.add(new Employee("Bob", 28));
        Collections.sort(list); // uses compareTo()
        for (Employee e : list) {
            System.out.println(e);
        }
    }  }
```

Feature	Comparable	Comparator
Purpose	Defines **natural ordering** of objects	Defines **custom ordering** of objects
Package	java.lang	java.util
Method	compareTo (T obj)	compare (T obj1, T obj2)
Where used	Inside the class	Outside the class
Modification required	Yes, class must implement Comparable	No modification needed in class
Number of sorting rules	Only **one**	Multiple sorting rules possible
Flexibility	Less flexible	Highly flexible
Used by	Collections.sort(), TreeSet, TreeMap	Collections.sort(), TreeSet, TreeMap

Generics

Generics = **Parameterized Types**
(meaning, the type is a **parameter** — just like passing a value to a function, you pass a **type** to a class or method)

Generics allow **one class, interface, or method** to **work with different types safely** — without writing multiple versions for every type.

In **old Java (before generics)**:

- People used **Object type** to create classes that could hold **any type**.

- But you had to **explicitly cast** when you got the data back.

- It was **unsafe** because you could cast wrong types and crash your program at runtime!

```
import java.util.*;
class OldBox {
        Object item;
        void set(Object item) {
        this.item = item;
        }
        Object get() {
        return item;
        }
}
```

```
class Main {
    public static void main(String args[]) {
    OldBox box = new OldBox();
    box.set("Hello");

    String s = (String) box.get(); // Need to cast
    System.out.println(s);

    box.set(123); // Now setting Integer
    String wrong = (String) box.get(); // Runtime Error! ClassCastException
    }
}
```

With Generics (Modern Java)

Generics fix this by **forcing type checking at compile time**.

Now you tell the class **what type** it should work with, like this: Box<String>, Box<Integer>, etc.

The General Form of a Generic Class

The generics syntax shown in the preceding examples can be generalized. Here is the syntax for declaring a generic class:

class class-name<type-param-list> { // ...

Here is the syntax for declaring a reference to a generic class:

class-name<type-arg-list> var-name =

new class-name<type-arg-list>(cons-arg-list);

```
// Generic class
class Box<T> {   // <T> means "type parameter T"
    T item;
    void set(T item) {
    this.item = item;
    }
    T get() {
    return item;
    }
}
```

```
class Main {

    public static void main(String args[]) {

    Box<String> stringBox = new Box<>();

    stringBox.set("Hello");

    String s = stringBox.get(); // No casting needed

    System.out.println(s);
```

- No runtime error.
- No manual type cast.
- Compiler catches mistakes early.

```
    Box<Integer> intBox = new Box<>();
    intBox.set(123);
    Integer i = intBox.get(); // Safe and correct
    System.out.println(i);
    // stringBox.set(123); // ❌ Error at compile time!
    }
}
```

Generics allow you to **write code once** and **use it with any type**, while **ensuring type safety**.

```
class Gen<T> {
```

// A simple generic class.

```
    T ob; // declare an object of type T
    Gen(T o) {
    ob = o;
    }
    T getob() {
    return ob;
    }
    void showType() {
System.out.println("Type of T is " + ob.getClass().getName());//runtime type (class name) of the object ob.
    }  }
```

```
class GenDemo {

    public static void main(String args[]) {

    // Create a Gen reference for Integer

    Gen<Integer> iOb = new Gen<>(88);           // Autoboxing int to Integer

    iOb.showType();                 // Output type info

    int v = iOb.getob();            // No cast needed

    System.out.println("value: " + v);

    System.out.println();
```

```
// Create a Gen reference for String
        Gen<String> strOb = new Gen<>("Generics Test");
        strOb.showType(); // Output type info
        String str = strOb.getob(); // No cast needed
        System.out.println("value: " + str);
        }
}
```

Output:

Type of T is java.lang.Integer

value: 88

Type of T is java.lang.String

value: Generics Test

Generics Work Only with Objects

In Java, generics allow you to define classes, interfaces, and methods that work with different data types while ensuring type safety. However, one important restriction is that generics only work with **reference types** (i.e., objects), and not with **primitive types** (like int, char, etc.).

Why Can't You Use Primitives Directly?

Generics require that the type parameter T is a reference type, not a primitive type. For example, the following code would result in a compilation error:

Gen<int> strOb = new Gen<int>(53); // Error!

Solution: Use Wrapper Classes

Java provides wrapper classes for primitive types to overcome this limitation. For instance:

- Integer for int
- Character for char
- Double for double
- Boolean for boolean

Thus, the corrected code would look like this:

Gen<Integer> iOb = new Gen<Integer>(53); // This works

Generic Types Differ Based on Their Type Arguments

Java's generics are **type-safe**—this means that a reference to one specific version of a generic type is **not compatible** with a reference to another version of the same generic type if the type arguments differ.

Example of Type Safety with Generics:

Consider this program with two versions of the Gen class:

Gen<Integer> iOb = new Gen<Integer>(53); // works for Integer

Gen<String> strOb = new Gen<String>("Hello"); // works for String

You cannot assign strOb (which is a Gen<String>) to iOb (which is a Gen<Integer>) because they are two **different** types of Gen:

iOb = strOb; // Error! **iOb is Gen<Integer>, and strOb is Gen<String>**

Generics ensure **type safety** by preventing situations where type mismatches could occur at runtime. By enforcing type compatibility at compile time, Java avoids situations where you might mistakenly assign incompatible types to each other.

How Generics Improve Type Safety

The introduction of **generics** in Java allows you to write type-safe code that eliminates the need for explicit type casts, making your programs more robust and easier to maintain. To understand the benefits, let's compare how **non-generic** and **generic** classes behave, and how generics ensure type safety at **compile time**, preventing **runtime errors**.

Non-Generic Version (Using Object)

In the non-generic version, we define a class NonGen that uses Object as the data type. The Object class is the root of all Java classes, so you can store any type of object in it. However, this comes with a few drawbacks:

1. **Explicit Casting is Required**: When retrieving an object from the NonGen class, you need to cast it back to the correct type.

2. **Potential Runtime Errors**: Since the compiler cannot check the type of data stored in NonGen objects, it’s possible to mistakenly assign incompatible types, which will only cause errors at runtime.

```
class NonGen {
    Object ob; // ob is now of type Object
    NonGen(Object o) {
    ob = o;
    }
    Object getob() {
    return ob;
    }
    void showType() {
    System.out.println("Type of ob is " + ob.getClass().getName());
    }
}
```

```
class NonGenDemo {
    public static void main(String args[]) {
    NonGen iOb;
    iOb = new NonGen(88); // Store Integer
    iOb.showType();
    int v = (Integer) iOb.getob(); // Cast needed
    System.out.println("value: " + v);
```

```
    NonGen strOb = new NonGen("Non-Generics Test"); // Store String
    strOb.showType();
    String str = (String) strOb.getob(); // Cast needed
    System.out.println("value: " + str);

    // This compiles, but is conceptually wrong
    iOb = strOb;
    v = (Integer) iOb.getob(); // Run-time error!
    }
}
```

Generic Version (Using Generics)

```
class Gen<T> {
    T ob;  // ob is of type T
    Gen(T o) {
    ob = o;
    }
    T getob() {
    return ob;
    }
    void showType() {
    System.out.println("Type of T is " + ob.getClass().getName());
    }    }
```

```
class GenDemo {
    public static void main(String args[]) {
    Gen<Integer> iOb = new Gen<>(88); // Store Integer
    iOb.showType();
    int v = iOb.getob();  // No cast needed, auto-unboxing
    System.out.println("value: " + v);
```

```
        Gen<String> strOb = new Gen<>("Generics Test"); // Store String
        strOb.showType();
        String str = strOb.getob();  // No cast needed
        System.out.println("value: " + str);

        // Compilation error: type mismatch
        // iOb = strOb;  // Error: incompatible types
        }
}
```

Key Benefits of the Generic Version

1. **No Need for Casting**:

    - With generics, the **correct type** is known at compile time. This means **no explicit casting** is required when retrieving the object.

Example:

```
int v = iOb.getob(); // No cast needed, auto-unboxing

String str = strOb.getob(); // No cast needed
```

2. Type Safety at Compile Time:

- The Java compiler checks that the types are consistent throughout your code. For example, the line iOb = strOb; will cause a **compile-time error** because iOb is a Gen<Integer>, and strOb is a Gen<String>. This prevents **runtime errors** like ClassCastException.

Example:

iOb = strOb; // Compile-time error: incompatible types

3. No Runtime Errors:

- Since the Java compiler knows the types, type mismatch errors (like attempting to cast a String to an Integer) are caught at **compile time**, preventing runtime errors.

A Generic Class with Two Type Parameters

```
class TwoGen<T, V> {
    T ob1;
    V ob2;
    TwoGen(T o1, V o2) {
    ob1 = o1;
    ob2 = o2;
    }
    void showTypes() {
    System.out.println("Type of T is " + ob1.getClass().getName());
    System.out.println("Type of V is " + ob2.getClass().getName());
    }
```

```
T getob1()
{
return ob1;
}
V getob2()
{
return ob2;
}
}
```

```
class SimpGen {
    public static void main(String args[]) {
        TwoGen<Integer, String> tgObj = new TwoGen<>(88, "Generics");
    // Show types
    tgObj.showTypes();
    // Get and print values
    int v = tgObj.getob1();
    System.out.println("value: " + v);
    String str = tgObj.getob2();
    System.out.println("value: " + str);
    }   }
```

Output:

Type of T is java.lang.Integer

Type of V is java.lang.String

value: 88

value: Generics

getClass().getName() prints the runtime type of each object.

T is replaced with Integer, and V with String.

Since generics enforce type safety, no casting is needed when retrieving values from tgObj.

What Are Bounded Types?

In Java Generics, **bounded types** restrict the types that can be passed to a type parameter. You can specify an **upper bound** using the extends keyword:

<T extends SomeClass>

This means T must be SomeClass or any of its subclasses (including classes that implement its interfaces if it's an interface).

Why Bounded Types?

Without bounds, generics accept *any* type. But sometimes, you only want to allow specific kinds of types—like only numeric types (Integer, Double, etc.), or types that implement a particular interface.

Example: Average of Numbers

Suppose you want to write a class to calculate the average of numeric values. If you don't bound the type, the compiler can't be sure the passed type supports numeric methods like doubleValue().

```
class Stats<T> {
        T[ ] nums;
        Stats(T[ ] o) {
        nums = o;
        }
        double average() {
        double sum = 0.0;
        for(int i=0; i < nums.length; i++)
                sum += nums[i].doubleValue(); // ❌ Compile-time error
        return sum / nums.length;
        }
}
```

Without Bound (This Fails)

Error: T could be *anything*, and the compiler can't guarantee doubleValue() exists.

```
public class BoundsDemo {
    public static void main(String[] args) {
    Integer[ ] intNums = {1, 2, 3, 4, 5};
    Stats<Integer> intStats = new Stats<>(intNums);
    System.out.println("Average (Integer): " + intStats.average());

    Double[ ] doubleNums = {1.1, 2.2, 3.3, 4.4};
    Stats<Double> doubleStats = new Stats<>(doubleNums);
    System.out.println("Average (Double): " + doubleStats.average());
    }
}
```

Without bounded types, T can be any object type, such as **String, Character, etc.** Those **classes do not contain doubleValue().**

So Java gives an error similar to:

cannot find symbol: method doubleValue()

```
class Stats<T extends Number> {
    T[ ] nums;
    Stats(T[ ] o) {
    nums = o;
    }
    double average() {
    double sum = 0.0;
    for(int i=0; i < nums.length; i++)
        sum += nums[i].doubleValue(); // ❌ Compile-time error
    return sum / nums.length;
    }
}
```

With Bounded Type

Error: T could be *anything*, and the compiler can't guarantee doubleValue() exists.

```
public class BoundsDemo {
    public static void main(String[] args) {
    Integer[ ] intNums = {1, 2, 3, 4, 5};
    Stats<Integer> intStats = new Stats<>(intNums);
    System.out.println("Average (Integer): " + intStats.average());
    Double[ ] doubleNums = {1.1, 2.2, 3.3, 4.4};
    Stats<Double> doubleStats = new Stats<>(doubleNums);
    System.out.println("Average (Double): " + doubleStats.average());
    // String[ ] strings = {"1", "2", "3"};
    // Stats<String> strStats = new Stats<>(strings); // ❌ Compile-time error
    }
}
```

Creating a Generic Method in Java

A **generic method** is a method that defines its own type parameters **independent** of any generic class. It allows the method to operate on objects of various types while maintaining type safety.

Syntax of a Generic Method:

```
<type-param-list> return-type method-name(parameter-list) {
    // method body
}
```

type-param-list: One or more **type parameters**, placed before the return type.

These type parameters can be **bounded** (using extends) or **unbounded**.

They are valid only within the scope of the method.

Example: Generic Method isIn()- This method checks if a given object exists inside an array.

```
class GenMethDemo {
    // Generic method
    static <T, V extends T> boolean isIn(T x, V[ ] y) {
    for (int i = 0; i < y.length; i++) {
        if (x.equals(y[i])) return true;
    }
    return false;
    }
```

T: The type of the item we're checking (e.g., Integer or String).

V: The type of the array elements. It must be the same as or a **subtype of T**.

This enforces **type compatibility** between the value and the array elements.

```
public static void main(String[] args) {
    // Integer array test
    Integer[ ] nums = {1, 2, 3, 4, 5};
    System.out.println(isIn(2, nums)); // true
    System.out.println(isIn(7, nums)); // false
    // String array test
    String[ ] words = {"apple", "banana", "cherry"};
    System.out.println(isIn("banana", words)); // true
    System.out.println(isIn("grape", words)); // false
    // This will NOT compile – incompatible types
    // System.out.println(isIn("banana", nums));
} }
```

Why Use Generic Methods?

- Code **reusability**: Write once, use with any type.
- **Type safety**: Compile-time type checking prevents bugs.
- Works **independently** of generic classes.

```
public class Util {
    // A method that returns the maximum of three comparable values
    public static <T extends Comparable<T>> T max(T a, T b, T c) {
    T max = a;
    if (b.compareTo(max) > 0) max = b;
    if (c.compareTo(max) > 0) max = c;
    return max;
    }
    public static void main(String[] args) {
    System.out.println(max(3, 7, 5));            // Integer
    System.out.println(max("apple", "banana", "kiwi")); // String
    }
}
```

T extends Comparable<T> ensures that the type can be compared using .compareTo().

Generic Constructors in Java

A **generic constructor** is a constructor that defines its own **type parameter**, independent of whether the class itself is generic.

This is useful when the constructor needs to accept different types in a type-safe manner, even if the class doesn't need to store or use that type generically elsewhere.

Syntax of a Generic Constructor:

```
class ClassName {
    // Generic constructor
    <T> ClassName(T param) {
    // constructor body
    }
}
```

The type parameter <T> is declared **before the constructor name**.

This parameter is **local to the constructor**, not the entire class.

Example: Generic Constructor with Number Types

```
// A non-generic class with a generic constructor
class GenCons {
        private double val;
        // Generic constructor with a bounded type parameter
        <T extends Number> GenCons(T arg) {
        val = arg.doubleValue(); // Use Number method
        }
        void showVal() {
        System.out.println("val: " + val);
        }
}
```

```
// Demo class
class GenConsDemo {
        public static void main(String[] args) {
        GenCons test = new GenCons(100);        // Integer
        GenCons test2 = new GenCons(123.5F); // Float

        test.showVal();   // Output: val: 100.0
        test2.showVal();   // Output: val: 123.5
        }
}
```

GenCons is a **non-generic class**.

Its constructor is declared as:

<T extends Number> GenCons(T arg)

This means:

- It accepts any object of type T where T **extends Number** (like Integer, Float, Double, etc.).

- Inside the constructor, arg.doubleValue() works because all subclasses of Number implement that method.

Benefits of Generic Constructors

Feature	Description
Type safety	Ensures only appropriate types (e.g., numbers) are used
Reusability	Works with multiple data types
Independence	Can be used in non-generic classes
Flexibility	You don’t need to make the whole class generic

Generic Interface

A generic interface in Java is an interface that works with a type parameter, so it can be reused for different data types while maintaining type safety.

Basic Syntax

```
interface InterfaceName<T> {
   void method(T data);
}
```

- <T> is a **type parameter**
- It can represent **any data type** (Integer, String, etc.)

Example 1: Simple Generic Interface

```
interface Printer<T>
{
   void print(T data);
}
class StringPrinter implements Printer<String>
 {
    public void print(String data) {
        System.out.println("String: " + data);
    }
}

class IntegerPrinter implements Printer<Integer>
 {
    public void print(Integer data) {
        System.out.println("Integer: " + data);
    }
}
```

```
public class Demo
{
public static void main(String[] args) {
Printer<String> p1 = new StringPrinter();
p1.print("Hello");

Printer<Integer> p2 = new IntegerPrinter();
p2.print(100);
}
}
```

Example 2: Generic Interface with Return Type

```
interface Calculator<T> {
   T add(T a, T b);
}
class IntegerCalculator implements Calculator<Integer> {
   public Integer add(Integer a, Integer b) {
       return a + b;
   }
}
class DoubleCalculator implements Calculator<Double> {
   public Double add(Double a, Double b) {
       return a + b;
   }
}
```

Example 3: Generic Interface Implemented by Generic Class

```
interface Storage<T> {
   void store(T item);
}
class DataStore<T> implements Storage<T> {
   public void store(T item) {
      System.out.println("Stored: " + item);
   }
}
public class Main {
   public static void main(String[] args) {
      DataStore<String> ds1 = new DataStore<>();
      ds1.store("Java");

      DataStore<Integer> ds2 = new DataStore<>();
      ds2.store(123);
   }
}
```

Programming Questions

Q1. Sort Employees by Age Using Comparator

Write a Java program to define a class Employee with fields String name and int age. Create a list of Employee objects. Implement a Comparator to sort the employees in ascending order of age. Print the list before and after sorting.

Q2. Sort Strings by Length

Write a Java program to sort a list of strings by their length using the Comparator interface. Display the list before and after sorting.

Q3. Generic Class with One Type Parameter

Write a generic class Box<T> with: A private member T value. A constructor to initialize the value. A method getValue() to return the value. Create instances of Box for Integer, String, and Double, and print their contents.

Q4. Generic Method to Swap Two Elements

Write a Java program with a generic method swapElements that swaps two elements of an array at given indices. Use this method to swap elements in: an Integer array AND a String array.

Q5. Generic Constructor

Create a class Printer that uses a **generic constructor** to print the value passed to it, regardless of its type. Create objects with different data types (e.g., int, String, double) and call the constructor.

Q6. Generic Class with Two Parameters

Write a generic class Pair<T, U> with: Two variables of type T and U, Constructor to initialize both, A method display() to print them. Create objects like:

Pair<Integer, String> p1 = new Pair<>(1, "One");

Pair<Double, Character> p2 = new Pair<>(3.14, 'A');

Q7. Write a Java program to create a class Employee with fields name and salary. Implement the Comparable interface to sort employees in **ascending order of salary**.

Q8. Create a list of integers and sort them in descending order by implementing Comparable in a wrapper class.

Q6. Sort Movies by Release Year Create a class Movie with name and year. Sort movies based on release year (latest first) using Comparable.

Q7. Sort Employees by Name Length Create a class Employee with name and id. Sort employees based on length of name using Comparable.

Q8. Sort Cities Alphabetically Create a class City with cityName and population. Sort cities alphabetically by name using Comparable.

Q9. Sort Bank Accounts by Balance Create a class Account with accountNumber and balance. Sort accounts in descending order of balance

Implement a generic class called **SortingUtility** that can sort a list of any type of object. The class should have a method called sortList that accepts two parameters: a List<T>, where T can be any type of object (such as a predefined class like String, Integer, or a custom class like Person), and a Comparator<T> that defines the sorting order for the elements in the list. To demonstrate the utility, you must implement at least one custom comparator to sort a list of Person objects. This comparator should allow sorting based on either the **age** or **name** of the Person objects. After sorting, the method should print the sorted list to the console, showing the results of the sorting operation.